claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a7b08445e4a4350b5.jsonl`
- SHA-256: `37a65e92ba5d618aac9557c952953c3f459b72cc01b84c5a8fc80cd4246f0675`
- Source modified: `2026-06-10T16:31:15+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:27:42.328Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Savings oracle ignores available opening/closing bookends when the balance walk passes ? missing leading/trailing transactions persist as 'reconciled'
SEVERITY CLAIMED: high
FILE: src/muneem/parsers/savings.py (line 78-84, 194-205)
DESCRIPTION: reconcile_oracle returns True as soon as the per-row balance walk passes, never checking the printed opening/closing even when parse_statement_summary found them. The walk only verifies CONSECUTIVE rows that are present, so any omission at the statement's edges is invisible: a dropped first/last transaction line (the exact clusterer failure class seen in Axis May-2025 ? it was only quarantined because the dropped line was interior), a leading/trailing extract_tables table that failed to map (parse_savings silently keeps only oracle-endorsed tables, then re-checks the combined set whose walk still passes), or a truncated multi-page PDF. Empirically confirmed: with opening=10000.0 available, the oracle endorsed (a) a ledger whose entire leading half was missing (implied opening 9000 != 10000) and (b) a first row whose debit was misread as 9999 instead of 500 ? the first balance-bearing row's flows are never compared against anything. The HDFC parser has the same blind spot via its fallback: when parse_summary returns None (e.g. the final page carrying the SUMMARY block is missing ? the very scenario that also truncates transactions), the verdict downgrades to reconcile(stmt).ok alone and grants RECONCILED. Note validate via verify() correctly withholds trust without bookends (test_verify_unavailable_figures...), but neither the savings path nor HDFC's fallback uses that discipline.
EVIDENCE QUOTED: savings.py:78-81: `def check(rows): if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok: return True` ? short-circuits before the opening/closing check. hdfc.py:449-453: `ok = (validate_against_summary(stmt, summary).ok if summary is not None else reconcile(stmt).ok)`. Empirical run: oracle(opening=10000, closing=8500) endorsed rows starting at balance 8000 with a 1000 debit (implied opening 9000) -> True; endorsed first-row debit=9999 against balance 9500 -> True.
SUGGESTED FIX: In reconcile_oracle, when opening is available require the implied opening (rows[0].balance + debit0 - credit0 == opening) and when closing is available require rows[-1].balance == closing, in ADDITION to the walk (i.e. reuse verify()'s bookend checks). Treat walk-pass-without-any-bookend as a distinct lower-confidence verdict rather than 'reconciled'. For HDFC, treat an absent SUMMARY as UNRECONCILED (verify() already does this) instead of falling back to the walk alone.

Your job is to try to REFUTE it. Your lens: existing-mitigation: check whether something ELSE in the repo (a test, a guard, a caller-side check, a hook, config) already prevents or detects the claimed problem.
Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/,

testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service. Structured output only.

2. user (2026-06-10T16:27:50.035Z)

```
1       """Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).
2
3       Every issuer whose statement is a transaction table follows the same recipe, so it lives
here once
4       and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:
5
6       1. **Extraction proposes** candidate cell-matrices ? one per ``extract_tables()`` table,
plus a
7          coordinate-clustered whole-statement matrix (for layouts ``extract_tables`` mis-
segments).
8       2. The generic **engine** maps each candidate's columns onto the canonical concepts.
9       3. **Reconciliation disposes:** a parse is trusted only if its per-row balance walks, or
? when a
10         vintage has no per-row balance ? the totals reconcile to the printed
``opening``/``closing``.
11
12      A statement splits its ledger across several extracted tables (page breaks), so we map
each table
13      on its own, keep the rows the oracle endorses, and re-check the *combined* set (which
catches a
14      cross-table balance break, i.e. a missing transaction). The result is **oracle-gated**:
a layout we
15      can't yet read returns ``verified=False`` and the caller persists nothing ? never
mislabeled rows.
16
17      No statement content is logged; only counts/booleans travel out.
18      """
19
20      from __future__ import annotations
21
22      import re
23      from dataclasses import dataclass, field
24      from pathlib import Path
25
26      import pdfplumber
27
28      from .clustering import cluster_to_matrix
29      from .concepts import ConceptRow, StatementConcepts
30      from .engine import Profile, map_table
31      from .validation import parse_amount, reconcile_running_balance, reconcile_total
32
33      _MONEY_TOKEN = re.compile(r"^-?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
34      _OPENING_RE = re.compile(r"opening\s+balance[^\d\-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
35      _CLOSING_RE = re.compile(r"closing\s+balance[^\d\-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
36
37
38      @dataclass
39      class SavingsParse:
40          """Outcome of a savings parse: concept rows + whether reconciliation endorsed
them."""
41
42          account_number: str
```

```
        rows: list[ConceptRow] = field(default_factory=list)
        verified: bool = False


    def parse_statement_summary(text: str) -> StatementConcepts:
        """Read opening/closing balance from statement text (for the total-reconcile
oracle).

        Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row
walk.
        """
        opening = _OPENING_RE.search(text)
        closing = _CLOSING_RE.search(text)
        return StatementConcepts(
            opening_balance=parse_amount(opening.group(1)) if opening else None,
            closing_balance=parse_amount(closing.group(1)) if closing else None,
        )


    def txn_line_predicate(profile: Profile):
        """A line is transaction-like if it carries one of the profile's dates *and* a money
token.

        Used to restrict coordinate clustering to the transaction band (drops headers /
summaries /
        address blocks that would otherwise wreck column inference).
        """
        patterns = profile.date_patterns

        def keep(texts: list[str]) -> bool:
            has_date = any(any(p.match(t) for p in patterns) for t in texts)
            return has_date and any(_MONEY_TOKEN.match(t) for t in texts)

        return keep


    def reconcile_oracle(opening: float | None, closing: float | None):
        """Trust a parse if the per-row balance walks, else if the totals reconcile to the
bookends."""

        def check(rows: list[ConceptRow]) -> bool:
            if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
                return True
            if rows and opening is not None and closing is not None:
                return reconcile_total(rows, opening, closing)
            return False

        return check


    def _reconstruct_by_balance_delta(
        matrix: list[list[str]], profile: Profile, opening: float | None, tol: float = 0.01
    ) -> list[ConceptRow]:
        """Recover debit/credit from the running balance when columns won't split cleanly.

        Needs only three things per row ? a date, the transaction amount, and the running
balance ? so
        it tolerates messy column inference (a sliver column, an interleaved value-date
column, or a
        single merged withdrawal/deposit column). Direction is the sign of the balance
change; the
        caller's reconciliation then **cross-checks** that the amount equals |?balance| on
every row, so
        a misread or a dropped row still fails (it is *not* circular). Needs the printed
opening balance
```

```
        to fix the first row's direction; returns ``[]`` if it can't form a clean series.
        """
        patterns = profile.date_patterns or (re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),)
        rows = [[(str(c).strip() if c else "") for c in r] for r in (matrix or []) if r]
        if not rows or opening is None:
            return []
        ncols = max(len(r) for r in rows)

        def is_date(s: str) -> bool:
            return any(p.match(s) for p in patterns)

        def money(s: str) -> float | None:
            return parse_amount(s) if _MONEY_TOKEN.match(s) else None

        # date column = the leftmost of the (first few) columns with the most date cells
        date_counts: dict[int, int] = {}
        for r in rows:
            for c in range(min(ncols, 3)):
                if c < len(r) and is_date(r[c]):
                    date_counts[c] = date_counts.get(c, 0) + 1
        if not date_counts:
            return []
        top = max(date_counts.values())
        date_col = min(c for c, n in date_counts.items() if n == top)

        txn_rows = [r for r in rows if date_col < len(r) and is_date(r[date_col])]
        if len(txn_rows) < 2:
            return []

        # A money column's money cells must outnumber its date cells ? so a sparsely-filled
deposit
        # column still counts, but an interleaved value-date column (mostly dates) does not.
The
        # running balance is the rightmost money column; the rest are the flow
(withdrawal/deposit).
        money_cols = []
        for c in range(ncols):
            m = sum(1 for r in txn_rows if c < len(r) and money(r[c]) is not None)
            d = sum(1 for r in txn_rows if c < len(r) and is_date(r[c]))
            if m >= 1 and m > d:
                money_cols.append(c)
        if len(money_cols) < 2:
            return []
        balance_col = max(money_cols)
        flow_cols = [c for c in money_cols if c != balance_col]

        text_cols = [c for c in range(ncols) if c != date_col and c not in money_cols]
        narr_col = (
            max(text_cols, key=lambda c: sum(len(r[c]) for r in txn_rows if c < len(r)))
            if text_cols
            else None
        )

        out: list[ConceptRow] = []
        prev = opening
        for r in txn_rows:
            bal = money(r[balance_col]) if balance_col < len(r) else None
            if bal is None:
                return []  # a running balance on every row is required for the delta
            amt = next((money(r[c]) for c in flow_cols if c < len(r) and money(r[c]) is not
None), None)
            debit = credit = None
            if amt is not None:
                if bal < prev - tol:
                    debit = amt
```

```
                elif bal > prev + tol:
                    credit = amt
                else:
                    debit = amt  # zero net with an amount present ? reconciliation will
flag it
            out.append(
                ConceptRow(
                    txn_date=r[date_col],
                    narration=(r[narr_col] if narr_col is not None and narr_col < len(r)
else ""),
                    debit=debit,
                    credit=credit,
                    balance=bal,
                )
            )
            prev = bal
        return out


    def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
        """Parse a savings statement into concept rows + a reconciliation verdict, per the
profile.

        Per-table mapping first (each extracted table mapped on its own; oracle-endorsed
rows combined
        and re-checked), then a coordinate-clustering fallback. Returns ``verified=False``
if neither
        reconciles, so the caller can refuse to persist.
        """
        with pdfplumber.open(pdf_path, password=password) as pdf:
            words_by_page = [
                page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
            ]
            tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
            head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))

        account_number = profile.first_account(head) or "UNKNOWN"
        summary = parse_statement_summary(head)
        oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)

        confirmed: list[ConceptRow] = []
        for table in tables:
            res = map_table(table, profile, oracle=oracle)
            if res and res.verified:
                confirmed.extend(res.rows)
        if confirmed and oracle(confirmed):
            return SavingsParse(account_number, confirmed, True)

        matrix = cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile))
        clustered = map_table(matrix, profile, oracle=oracle)
        if clustered and clustered.verified:
            return SavingsParse(account_number, clustered.rows, True)

        # 3) balance-delta reconstruction ? recovers vintages whose columns don't split into
a clean
        #    debit/credit (merged amount column, interleaved value-date, sliver columns):
derive the
        #    direction from the running balance and let the oracle cross-check amount ==
|?balance|.
        delta = _reconstruct_by_balance_delta(matrix, profile, summary.opening_balance)
        if delta and oracle(delta):
            return SavingsParse(account_number, delta, True)
```

```
213
214         fallback = confirmed or (clustered.rows if clustered else [])
215         return SavingsParse(account_number, fallback, False)
216
217
218     __all__ = [
219         "SavingsParse",
220         "parse_savings",
221         "parse_statement_summary",
222         "txn_line_predicate",
223         "reconcile_oracle",
224     ]
225
```

3. user (2026-06-10T16:27:50.857Z)

```
src\muneem\parsers\__init__.py
src\muneem\parsers\concepts.py
src\muneem\parsers\validation.py
src\muneem\parsers\clustering.py
src\muneem\parsers\profiles.py
src\muneem\parsers\savings.py
src\muneem\parsers\engine.py
src\muneem\parsers\base.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\hdfc.py
```

4. user (2026-06-10T16:28:01.537Z)

```
1       """The **verifier** ? reconciliation as a layout-independent truth oracle.
2
3       Reconciliation proves a parse is **complete and correct without inspecting any sensitive
value**:
4       if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
us trust a
5       parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
safe on
6       money ? *extraction proposes, reconciliation disposes* (see
7       [`docs/parser-architecture.md`](../../../docs/parser-architecture.md) §2, §5).
8
9       This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11      * **per-row walk** ? ``reconcile_running_balance``: ``balance[i] == balance[i-1] ? debit
+ credit``.
12      * **total reconcile** ? ``reconcile_total``: ``opening + ?credit ? ?debit == closing``
(works even
13        with no per-row balance, e.g. Axis savings).
14      * **bookend** ? ``verify``: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15      * **selection** ? ``select_reconciling``: among candidate concept mappings, pick the one
arithmetic
16        endorses (the seam the L2 mapping search plugs into).
17      * **confidence** ? ``Confidence`` + ``BookendReport.confidence``: a recorded verdict,
not an
18        assumption.
19
20      Reports carry counts / booleans / a verdict only ? **never an amount**.
21      """
22
23      from __future__ import annotations
24
25      import re
26      from collections.abc import Sequence
```

```
    from dataclasses import dataclass
    from enum import StrEnum

    from .concepts import ConceptRow, StatementConcepts

    _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")


    def parse_amount(value: object) -> float | None:
        """Parse a statement money cell to a float.

        Returns ``None`` only for a genuinely absent cell (None / empty / ``"-"`` /
        non-numeric); ``"0.00"`` parses to ``0.0``. This deliberately distinguishes
        *absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"`` to
        ``None`` and would corrupt a real zero balance and the reconciliation math.
        """
        if value is None:
            return None
        text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
        if text in ("", "-"):
            return None
        try:
            return float(text)
        except ValueError:
            return None


    @dataclass
    class ReconcileReport:
        """Result of a running-balance walk. Counts only, never an amount."""

        n: int  # rows considered
        compared: int  # consecutive (balance-bearing) pairs actually checked
        breaks: int
        first_break: int | None = None

        @property
        def ok(self) -> bool:
            return self.compared > 0 and self.breaks == 0


    def reconcile_running_balance(
        entries: Sequence[tuple[float | None, float | None, float | None]],
        tol: float = 0.01,
    ) -> ReconcileReport:
        """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.

        ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
        ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
        is skipped (can't anchor on it). Because the walk spans the whole statement, a
        clean result also rules out any row missed *between* two balances ? which is the
        failure mode silent ``extract_tables()`` row-drops would otherwise hide.
        """
        breaks = 0
        first_break: int | None = None
        compared = 0
        prev: float | None = None
        for i, (debit, credit, balance) in enumerate(entries):
            if balance is None:
                continue
            if prev is not None:
                expected = prev - (debit or 0.0) + (credit or 0.0)
                compared += 1
                if abs(expected - balance) > tol:
                    breaks += 1
```

```
                    if first_break is None:
                        first_break = i
            prev = balance
        return ReconcileReport(
            n=len(entries), compared=compared, breaks=breaks, first_break=first_break
        )


    def _row_entries(
        rows: Sequence[ConceptRow],
    ) -> list[tuple[float | None, float | None, float | None]]:
        """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk consumes."""
        return [(r.debit, r.credit, r.balance) for r in rows]


    def reconcile_total(
        rows: Sequence[ConceptRow], opening: float, closing: float, tol: float = 0.01
    ) -> bool:
        """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.

        This is the oracle for statements with **no reliable per-row balance** (e.g. Axis savings):
        it checks the whole statement against its own opening/closing bookends without needing each
        row's running balance. ``opening`` and ``closing`` come from the bank's SUMMARY/header
        (``StatementConcepts``); the caller must have them.
        """
        delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
        return abs((opening + delta) - closing) <= tol


    class Confidence(StrEnum):
        """Recorded trust verdict for a parsed statement (mirrors ``documents.status`` values).

        ``RECONCILED`` ? the parse passed every available check, so it is trusted.
        ``UNRECONCILED`` ? a check failed (or none could run); the parse is suspect and is quarantined rather than trusted.
        """

        RECONCILED = "reconciled"
        UNRECONCILED = "unreconciled"


    @dataclass
    class BookendReport:
        """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.

        ``None`` for a check means the figure wasn't available to verify (not a failure). ``ok``
        requires the per-row balance walk *plus* every available check to pass.
        """

        n: int
        reconciles: bool
        debit_count_ok: bool | None = None
        credit_count_ok: bool | None = None
        opening_ok: bool | None = None
        closing_ok: bool | None = None

        @property
        def ok(self) -> bool:
```

```
            checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
self.closing_ok]
            return self.n > 0 and self.reconciles and all(c is True for c in checks)

        @property
        def confidence(self) -> Confidence:
            return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED


    def verify(
        rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
    ) -> BookendReport:
        """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
figures.

        The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
any
        extra/missing row; the opening & closing balances catch a missing first/last row.
Each printed
        figure is only checked when present (``None`` ? not checked). The opening balance is
checked by
        implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
opening).
        """
        reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
        debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
        if concepts.debit_count is not None:
            debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
concepts.debit_count
        if concepts.credit_count is not None:
            credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
concepts.credit_count
        if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
            closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
        if rows and rows[0].balance is not None and concepts.opening_balance is not None:
            implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
0.0)
            opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
        return BookendReport(
            n=len(rows),
            reconciles=reconciles,
            debit_count_ok=debit_count_ok,
            credit_count_ok=credit_count_ok,
            opening_ok=opening_ok,
            closing_ok=closing_ok,
        )


    def select_reconciling(candidates: Sequence[Sequence[ConceptRow]], tol: float = 0.01) ->
int | None:
        """Among candidate concept-row mappings, return the index of the first that
reconciles.

        This is the *selection* half of "extraction proposes, reconciliation disposes": when
the
        extraction ladder can't be sure which column?concept assignment is right, it
proposes several
        and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.

        The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
        the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
        running balance ? balance-less statements select via :func:`reconcile_total` at the
call site.
```

```
        """
        for i, rows in enumerate(candidates):
            if reconcile_running_balance(_row_entries(rows), tol).ok:
                return i
        return None


    __all__ = [
        "parse_amount",
        "ReconcileReport",
        "reconcile_running_balance",
        "reconcile_total",
        "Confidence",
        "BookendReport",
        "verify",
        "select_reconciling",
    ]

```

5. user (2026-06-10T16:28:02.072Z)

```
                    "narration",
                    "ref",
                    "withdrawal",
                    "deposit",
                    "closing_balance",
                    "page",
                ]
            )
            for t in statement.transactions:
                writer.writerow(
                    [
                        t.date,
                        t.value_date or "",
                        t.narration,
                        t.ref or "",
                        f"{t.withdrawal:.2f}",
                        f"{t.deposit:.2f}",
                        f"{t.closing_balance:.2f}",
                        t.page,
                    ]
                )
        return out


    @register_parser
    class HDFCParser(StatementParser):
        """Registered parser for HDFC savings / transaction-account statements."""

        def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
            low = (text or "").lower()
            is_hdfc = "hdfc bank" in low or "hdfcbank" in low or "hdfcbank" in (sender or
"").lower()
            is_statement = "savings account details" in low or (
                "narration" in low and "closing balance" in low
            )
            return is_hdfc and is_statement

        def parse_and_insert(
            self,
            pdf_path: Path,
            conn,
            doc_id: int,
            password: str | None = None,
```

```
422         ) -> int:
423             """Parse, insert rows into ``hdfc_transactions``, and record parse quality.
424
425             Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
426             balance walk (and the bank's SUMMARY when present) so a downstream consumer
427             can tell a trustworthy parse from a suspect one. Returns the row count.
428             """
429             try:
430                 stmt = parse_statement(pdf_path, password=password or "")
431             except ParseError:
432                 raise
433             except Exception as exc:
434                 raise ParseError(f"failed to parse HDFC statement
{redact_filename(pdf_path)}") from exc
435             for t in stmt.transactions:
436                 dbmod.insert_hdfc_transaction(
437                     conn,
438                     document_id=doc_id,
439                     account_number=stmt.account_number,
440                     txn_date=t.date,
441                     value_date=t.value_date,
442                     particulars=t.narration,
443                     ref=t.ref,
444                     withdrawal=t.withdrawal,
445                     deposit=t.deposit,
446                     closing_balance=t.closing_balance,
447                 )
448             summary = parse_summary(pdf_path, password=password or "")
449             ok = (
450                 validate_against_summary(stmt, summary).ok
451                 if summary is not None
452                 else reconcile(stmt).ok
453             )
454             verdict = Confidence.RECONCILED if ok else Confidence.UNRECONCILED
455             dbmod.record_parse(
456                 conn, doc_id, confidence=verdict.value, parser="HDFCParser",
parser_version=__version__
457             )
458             return len(stmt.transactions)
459
```

6. user (2026-06-10T16:28:14.533Z)

```
1       """Axis Bank statement parsers: savings/transaction account + credit card.
2
3       **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
table +
4       coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
5       vintage it can't read stores nothing (``unreconciled``) rather than mislabeled rows.
Axis specifics
6       live entirely in ``AXIS_PROFILE``; this replaced the old fixed-column ``extract_tables``
mapping.
7
8       Credit-card statements carry no running balance, so ``AxisCreditCardParser`` can't
reconcile
9       (parse/persist split + count only). No statement content is logged here.
10      """
11
12      from __future__ import annotations
13
14      import re
15      import sqlite3
16      from dataclasses import dataclass
17      from pathlib import Path
```

```
18
19      import pdfplumber
20
21      from muneem import __version__
22      from muneem import db as dbmod
23      from muneem.errors import ParseError
24      from muneem.redaction import redact_filename
25
26      from . import register_parser
27      from .base import StatementParser
28      from .profiles import AXIS_PROFILE
29      from .savings import SavingsParse, parse_savings, parse_statement_summary,
txn_line_predicate
30      from .validation import Confidence, parse_amount
31
32      _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}")  # Axis credit card: dd/mm/yyyy
33      _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*+\d{4})")]
34
35
36      def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
37          for rx in patterns:
38              m = rx.search(text)
39              if m:
40                  return m.group(1).strip()
41          return None
42
43
44      # --- Savings / transaction account (concept core)
---------------------------------------------
45
46
47      # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
48      AxisSavings = SavingsParse
49      parse_axis_summary = parse_statement_summary
50      _is_txn_line = txn_line_predicate(AXIS_PROFILE)
51
52
53      def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
54          """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
55          return parse_savings(pdf_path, AXIS_PROFILE, password)
56
57
58      @register_parser
59      class AxisParser(StatementParser):
60          def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61              if sender and "axisbank.com" in sender.lower():
62                  return True
63              t = text or ""
64              return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66          def parse_and_insert(
67              self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68          ) -> int:
69              """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71              Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72              parser can't yet read surfaces loudly instead of saving mislabeled rows.
73              """
74              try:
```

```
                parsed = parse_axis_savings(pdf_path, password)
            except ParseError:
                raise
            except Exception as exc:
                # Filename only ? never statement text; original cause kept via `from`.
                name = redact_filename(pdf_path)
                raise ParseError(f"failed to parse Axis statement {name}") from exc

            if not parsed.verified:
                dbmod.record_parse(
                    conn,
                    doc_id,
                    confidence=Confidence.UNRECONCILED.value,
                    parser="AxisParser",
                    parser_version=__version__,
                )
                return 0

            for r in parsed.rows:
                dbmod.insert_axis_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=parsed.account_number,
                    txn_date=r.txn_date,
                    value_date=r.value_date or "",
                    particulars=r.narration,
                    debit=r.debit,
                    credit=r.credit,
                    balance=r.balance,
                )
            dbmod.record_parse(
                conn,
                doc_id,
                confidence=Confidence.RECONCILED.value,
                parser="AxisParser",
                parser_version=__version__,
            )
            return len(parsed.rows)


    # --- Credit card (no running balance; count only)
-------------------------------------------


    @dataclass
    class AxisCCTxn:
        txn_date: str
        particulars: str
        merchant_category: str
        amount: float
        type: str  # "Cr" or "Dr"


    def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
        """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
        rows: list[AxisCCTxn] = []
        for table in tables or []:
            for row in table or []:
                if not row or len(row) < 9:
                    continue
                date_str = str(row[0]).strip() if row[0] else ""
                if not _CC_DATE_RE.match(date_str):
                    continue
                amount_str = str(row[8]).strip() if row[8] else ""
                is_cr = "Cr" in amount_str
```

```
                amount = parse_amount(re.sub(r"[^\d.]", "", amount_str)) or 0.0
                rows.append(
                    AxisCCTxn(
                        txn_date=date_str,
                        particulars=str(row[2]).strip() if row[2] else "",
                        merchant_category=str(row[7]).strip() if row[7] else "",
                        amount=amount,
                        type="Cr" if is_cr else "Dr",
                    )
                )
        return rows


    def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
        """Open an Axis credit-card statement and return (card_number, transactions)."""
        card_number = "UNKNOWN"
        rows: list[AxisCCTxn] = []
        with pdfplumber.open(pdf_path, password=password) as pdf:
            head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
            card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
            for page in pdf.pages:
                rows.extend(_rows_from_cc_tables(page.extract_tables()))
        return card_number, rows


    @register_parser
    class AxisCreditCardParser(StatementParser):
        def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
            return "Axis Bank" in text and "Credit Card Statement" in text

        def parse_and_insert(
            self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
        ) -> int:
            try:
                card_number, rows = parse_axis_cc(pdf_path, password)
            except ParseError:
                raise
            except Exception as exc:
                raise ParseError(
                    f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
                ) from exc
            for t in rows:
                dbmod.insert_axis_cc_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=card_number,
                    txn_date=t.txn_date,
                    particulars=t.particulars,
                    merchant_category=t.merchant_category,
                    amount=t.amount,
                    type=t.type,
                )
            # No running balance to reconcile; documents.status stays as the caller set it.
We still
            # record provenance (with no confidence verdict).
            dbmod.record_provenance(
                conn,
                document_id=doc_id,
                source_type="pdf_statement",
                parser="AxisCreditCardParser",
                parser_version=__version__,
```

```
199                 mapping_layer="deterministic",
200             )
201             return len(rows)
202
```

7. user (2026-06-10T16:28:14.655Z)

```
1       """AU Small Finance Bank savings / transaction-account statement parser.
2
3       Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
4       coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
5       vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
entirely in
6       ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged
here.
7       """
8
9       from __future__ import annotations
10
11      import sqlite3
12      from pathlib import Path
13
14      from muneem import __version__
15      from muneem import db as dbmod
16      from muneem.errors import ParseError
17      from muneem.redaction import redact_filename
18
19      from . import register_parser
20      from .base import StatementParser
21      from .profiles import AUBANK_PROFILE
22      from .savings import SavingsParse, parse_savings
23      from .validation import Confidence
24
25
26      def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
27          """Parse an AU Bank savings statement via the shared engine-driven path."""
28          return parse_savings(pdf_path, AUBANK_PROFILE, password)
29
30
31      @register_parser
32      class AUParser(StatementParser):
33          def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
34              if sender and "aubank" in sender.lower():
35                  return True
36              low = (text or "").lower()
37              return "au small finance bank" in low or "aubank" in low
38
39          def parse_and_insert(
40              self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
41          ) -> int:
42              """Persist a savings parse **only if it reconciles**; otherwise store
nothing."""
43              try:
44                  parsed = parse_au_savings(pdf_path, password)
45              except ParseError:
46                  raise
47              except Exception as exc:
48                  raise ParseError(f"failed to parse AU statement
{redact_filename(pdf_path)}") from exc
49
50              if not parsed.verified:
51                  dbmod.record_parse(
```

```
                    conn,
                    doc_id,
                    confidence=Confidence.UNRECONCILED.value,
                    parser="AUParser",
                    parser_version=__version__,
                )
                return 0

            for r in parsed.rows:
                dbmod.insert_au_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=parsed.account_number,
                    txn_date=r.txn_date,
                    value_date=r.value_date or "",
                    particulars=r.narration,
                    debit=r.debit,
                    credit=r.credit,
                    balance=r.balance,
                )
            dbmod.record_parse(
                conn,
                doc_id,
                confidence=Confidence.RECONCILED.value,
                parser="AUParser",
                parser_version=__version__,
            )
            return len(parsed.rows)

```

8. user (2026-06-10T16:28:23.724Z)

```
      """The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.

      This is the *generic-first* half of the concept-core architecture (see
      [`docs/parser-architecture.md`](../../../docs/parser-architecture.md) §4, §6).
      Bank-agnostic code that takes a **cell-matrix** (the shape `pdfplumber.extract_tables()`
returns,
      or what a coordinate-clustering extractor emits) and maps its columns onto the canonical
concepts
      via a cheap-first ladder, with a profile supplying priors:

      * **L0 ? header lexicon.** Match header-row labels against the profile's concept?label
aliases.
      * **profile-order.** If there is no header, fall back to the profile's known column
order (a prior).
      * **L1/L2 ? content + search.** Classify columns by content (dates, money), then
enumerate the
        plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
        reconciliation oracle endorses** ? the self-identifying balance column, found by
*which mapping
        makes the math work* rather than by its label.

      The engine is **family-agnostic**: it is parameterized by a :class:`FamilySpec` (the
target concept
      names, which are date/amount/balance, and a typed-row builder) and an injected `oracle`
      (`rows -> bool`). Only the *bank* family ships today; an instrument family
(equity/MF/ETF/bond ? see
      the project scope) would reuse this control flow with its own concepts + oracle. Nothing
here logs a
      value; provenance is the winning layer + column map.
      """

      from __future__ import annotations

```

```
    import itertools
    import re
    from collections.abc import Callable, Sequence
    from dataclasses import dataclass, field

    from .concepts import ConceptRow
    from .validation import parse_amount

    Cell = object  # a table cell: str | None (we coerce defensively)
    Row = Sequence[Cell]

    # Generic Indian-statement date shapes; a profile can pin its own to avoid false
positives.
    _GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (
        re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"),  # dd/mm/yyyy or dd-mm-yyyy
        re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"),  # dd-Mon-yyyy / dd Mon yy
    )
    # "Money" requires a decimal point so bare integers (e.g. a long reference number) are
NOT mistaken
    # for an amount column during content classification.
    _MONEY_RE = re.compile(r"(?i)^-?[\d,]+\.\d{1,2}(?:\s*(?:cr|dr))?$")
    _BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")

    # The generic bank lexicon. Profiles extend/override per issuer; keys are concept names,
values are
    # lower-cased header labels matched exactly (not by substring) during L0.
    _BANK_ALIASES: dict[str, tuple[str, ...]] = {
        "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
        "value_date": ("value date", "val date", "value dt"),
        "narration": (
            "narration",
            "particulars",
            "description",
            "transaction details",
            "details",
            "remarks",
            "transaction",
            "transaction remarks",
        ),
        "reference": (
            "ref",
            "reference",
            "ref no",
            "reference no",
            "cheque no",
            "chq no",
            "instrument no",
        ),
        "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
        "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
        "balance": (
            "balance",
            "closing balance",
            "running balance",
            "closing bal",
            "available balance",
        ),
    }


    def _text(cell: Cell) -> str:
        return "" if cell is None else str(cell).strip()


    def _norm(label: str) -> str:
```

```
        return re.sub(r"\s+", " ", label.strip().lower())


    def _matches_date(text: str, patterns: Sequence[re.Pattern]) -> bool:
        return any(p.match(text) for p in patterns)


    def _looks_like_money(text: str) -> bool:
        return bool(_MONEY_RE.match(text))


    @dataclass
    class FamilySpec:
        """A concept *family*: the target vocabulary + how to build typed rows from a column
mapping.

        The engine's L0/L1/L2 logic is written against this spec rather than against bank
concepts, so a
        future instrument family (with its own concepts + reconciliation oracle) reuses the
same engine.
        ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
understands.
        """

        name: str
        concepts: tuple[str, ...]
        date_concepts: tuple[str, ...]  # in order: the first is the primary (row-gating)
date
        amount_concepts: tuple[str, ...]  # money columns; the non-balance ones are the
"flows"
        balance_concept: str | None  # the running-balance concept, if the family has one
        text_concepts: tuple[str, ...]  # free-text columns; the first gets the widest text
column
        default_aliases: dict[str, tuple[str, ...]]
        build: Callable[[list[dict[str, str]]], list]


    def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
        """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays
``None``)."""
        rows: list[ConceptRow] = []
        for m in mapped:
            rows.append(
                ConceptRow(
                    txn_date=_text(m.get("txn_date")),
                    narration=_text(m.get("narration")),
                    debit=parse_amount(m.get("debit")),
                    credit=parse_amount(m.get("credit")),
                    balance=parse_amount(m.get("balance")),
                    value_date=_text(m.get("value_date")) or None,
                    reference=_text(m.get("reference")) or None,
                )
            )
        return rows


    BANK_FAMILY = FamilySpec(
        name="bank",
        concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit",
"balance"),
        date_concepts=("txn_date", "value_date"),
        amount_concepts=("debit", "credit", "balance"),
        balance_concept="balance",
        text_concepts=("narration", "reference"),
        default_aliases=_BANK_ALIASES,
```

```
        build=_build_bank_rows,
    )


    @dataclass
    class MappingResult:
        """The outcome of mapping one table: typed rows + how we got there (provenance)."""

        rows: list  # typed rows from family.build (e.g. list[ConceptRow])
        mapped: list[
            dict[str, str]
        ]  # per-row {concept: raw cell}, incl. any profile-declared extra columns
        column_map: dict[str, int]  # the winning concept -> column-index assignment
        layer: str  # "L0" / "profile-order" / "L2" ? which rung produced it
        verified: bool  # did the oracle endorse it? (False if no oracle was supplied)


    @dataclass
    class Profile:
        """Declarative per-issuer knowledge ? *data, not code* (architecture §6).

        Generic detectors run first; the profile supplies priors: extra concept?label
aliases, the
        accepted date shape, a positional fallback when there's no header, table anchors,
account-number
        patterns, quirks, and (placeholder, wiring parked) the password rule.
        """

        name: str
        aliases: dict[str, tuple[str, ...]] = field(default_factory=dict)
        date_patterns: tuple[re.Pattern, ...] = ()
        column_order: tuple[str, ...] | None = None
        table_anchors: tuple[str, ...] = ()
        account_patterns: tuple[re.Pattern, ...] = ()
        quirks: frozenset[str] = frozenset()
        password_rule: str | None = None  # data only; sourcing is parked (DESIGN §8.2)

        def first_account(self, text: str) -> str | None:
            for rx in self.account_patterns:
                m = rx.search(text)
                if m:
                    return m.group(1).strip()
            return None


    def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
        """Build ``normalized-label -> concept`` (profile entries override family
defaults)."""
        index: dict[str, str] = {}
        for concept, labels in family.default_aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        for concept, labels in profile.aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        return index


    def _date_patterns(profile: Profile) -> tuple[re.Pattern, ...]:
        return profile.date_patterns or _GENERIC_DATE_PATTERNS


    def _detect_header(
        rows: list[list[Cell]], alias_index: dict[str, str]
    ) -> tuple[int | None, dict[str, int]]:
```

```
        """Find the header row (most exact alias hits, >=2) and its concept->column map."""
        best_idx: int | None = None
        best_map: dict[str, int] = {}
        for i, row in enumerate(rows):
            cmap: dict[str, int] = {}
            for col, cell in enumerate(row):
                concept = alias_index.get(_norm(_text(cell)))
                if concept and concept not in cmap:
                    cmap[concept] = col
            if len(cmap) > len(best_map):
                best_map, best_idx = cmap, i
        if len(best_map) < 2:
            return None, {}
        return best_idx, best_map


    def _is_balance_marker(row: Row) -> bool:
        return any(any(mk in _norm(_text(c)) for mk in _BALANCE_MARKERS) for c in row)


    def _is_transaction_row(row: Row, date_col: int | None, patterns: Sequence[re.Pattern])
-> bool:
        if date_col is not None:
            cell = row[date_col] if date_col < len(row) else None
            return _matches_date(_text(cell), patterns)
        return any(_matches_date(_text(c), patterns) for c in row)


    def _classify_columns(
        data: list[list[Cell]], ncols: int, patterns: Sequence[re.Pattern]
    ) -> dict[int, str]:
        """Tag each column date / amount / text / empty by majority content."""
        kinds: dict[int, str] = {}
        for c in range(ncols):
            cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
            if not cells:
                kinds[c] = "empty"
                continue
            thresh = max(1, int(len(cells) * 0.6))
            if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
                kinds[c] = "date"
            elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
                kinds[c] = "amount"
            else:
                kinds[c] = "text"
        return kinds


    def _base_map(
        col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
    ) -> dict[str, int]:
        """The unambiguous part of a content (L1) mapping: date columns + the primary text
column.

        Family-agnostic: date columns fill ``family.date_concepts`` in order; the widest
text column
        goes to the family's first text concept (if any). Amount columns are left to the
search.
        """
        base: dict[str, int] = {}
        date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
        text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
        for concept, col in zip(family.date_concepts, date_cols, strict=False):
            base[concept] = col
        if text_cols and family.text_concepts:
```

```
            base[family.text_concepts[0]] = max(
                text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
            )
        return base


    def _search_candidates(
        base: dict[str, int], col_kinds: dict[int, str], family: FamilySpec
    ) -> list[tuple[str, dict[str, int]]]:
        """Enumerate plausible amount-column ? concept assignments (the L2 search space).

        Family-agnostic: the "flows" are the family's amount concepts minus its balance
concept (for
        banks: debit, credit). We try each amount column as the balance and assign the rest
to the
        flows, then also the no-per-row-balance case (total-reconcile families, e.g. Axis
Oct).
        """
        amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]
        flows = [c for c in family.amount_concepts if c != family.balance_concept]
        bal = family.balance_concept
        cands: list[dict[str, int]] = []
        if bal and len(amount_cols) >= len(flows) + 1:
            for bcol in amount_cols:
                rest = [c for c in amount_cols if c != bcol]
                for assign in itertools.permutations(rest, len(flows)):
                    cands.append({**base, **dict(zip(flows, assign, strict=True)), bal:
bcol})
        if flows and len(amount_cols) >= len(flows):
            for assign in itertools.permutations(amount_cols, len(flows)):
                cands.append({**base, **dict(zip(flows, assign, strict=True))})
        return [("L2", c) for c in cands[:24]]


    def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
        out: list[dict[str, str]] = []
        for row in data:
            out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in
col_map.items()})
        return out


    def map_table(
        table: Sequence[Row] | None,  # Sequence (covariant) so list[list[str | None]]
callers fit
        profile: Profile,
        family: FamilySpec = BANK_FAMILY,
        *,
        oracle: Callable[[list], bool] | None = None,
    ) -> MappingResult | None:
        """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among
candidates.

        Resolution order: header lexicon (L0) ? profile column order ? content search (L2).
With an
        ``oracle`` the first candidate it endorses wins (``verified=True``); without one,
the first
        structurally-complete candidate is returned unverified (the caller reconciles
separately).
        Returns ``None`` if no transaction rows are found, or the best unverified candidate
if an oracle
        was supplied but nothing reconciled.
        """
        if not table:
            return None
```

```
320         rows_in = [list(r) for r in table if r is not None]
321         if not rows_in:
322             return None
323         alias_index = _alias_index(profile, family)
324         patterns = _date_patterns(profile)
325 
326         header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
327         start = header_idx + 1 if header_idx is not None else 0
328         date_col_hint = col_map_hdr.get("txn_date")
329         if date_col_hint is None and profile.column_order and "txn_date" in
profile.column_order:
330             date_col_hint = profile.column_order.index("txn_date")
331 
332         data = [
333             r
334             for r in rows_in[start:]
335             if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
336         ]
337         if not data:
338             return None
339         ncols = max(len(r) for r in data)
340         col_kinds = _classify_columns(data, ncols, patterns)
341         base = _base_map(col_kinds, data, family)
342         flows = [c for c in family.amount_concepts if c != family.balance_concept]
343 
344         candidates: list[tuple[str, dict[str, int]]] = []
345         if flows and all(c in col_map_hdr for c in flows):  # L0: header pinned the flow
concepts
346             candidates.append(("L0", {**base, **col_map_hdr}))
347         elif col_map_hdr:  # header present but didn't pin the flows: keep its hints, search
the rest
348             candidates.append(("L0", {**base, **col_map_hdr}))
349         if profile.column_order:  # positional prior (no/partial header)
350             candidates.append(("profile-order", {c: i for i, c in
enumerate(profile.column_order)}))
351         candidates += _search_candidates(base, col_kinds, family)
352 
353         primary_date = family.date_concepts[0] if family.date_concepts else None
354         best: MappingResult | None = None
355         for layer, cmap in candidates:
356             if primary_date and primary_date not in cmap:
357                 continue
358             if not (set(family.amount_concepts) & cmap.keys()):
359                 continue
360             mapped = _apply_map(data, cmap)
361             typed = family.build(mapped)
362             if oracle is None:
363                 return MappingResult(typed, mapped, cmap, layer, verified=False)
364             if oracle(typed):
365                 return MappingResult(typed, mapped, cmap, layer, verified=True)
366             if best is None:
367                 best = MappingResult(typed, mapped, cmap, layer, verified=False)
368         return best
369 
370 
371     __all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]
372 
```

9. user (2026-06-10T16:28:25.382Z)

```
252-    for line in lines:
253-        texts = {w["text"] for w in line}
254-        if "Narration" in texts and any(t.startswith("Closing") for t in texts):
255-            header_top = _line_top(line)
256-            break
```

```
257-    if header_top is None:
258-        return []
259-    return [line for line in lines if _line_top(line) > header_top + 1]
260-
261-
262-def _extract_account_number(text: str) -> str | None:
263-    """Account number via the HDFC profile (bank knowledge as data; same pattern as
before)."""
264-    return HDFC_PROFILE.first_account(text)
265-
266-
267:def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
268-    """Parse an HDFC statement PDF into a :class:`Statement` of transactions.
269-
270-    ``password`` is forwarded to pdfplumber for encrypted statements and used
271-    in-memory only ? never stored or logged.
272-    """
273-    path = Path(pdf_path)
274-    transactions: list[Transaction] = []
275-    account_number: str | None = None
276-    with pdfplumber.open(str(path), password=password or "") as pdf:
277-        for page_no, page in enumerate(pdf.pages):
278-            if account_number is None and page_no < 2:
279-                account_number = _extract_account_number(page.extract_text() or "")
280-            words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
281-            body = _table_body(_group_lines(words))
282-            transactions.extend(extract_transactions_from_lines(body, page_no))
283-    return Statement(path=path, transactions=transactions, account_number=account_number)
284-
285-
286:def reconcile(statement: Statement, tol: float = 0.01) -> ReconcileReport:
287-    """Check the running balance via the shared reconciler (closing[i] ==
288-    closing[i-1] - withdrawal + deposit).
289-
290-    Spans page breaks (transactions are in document order), so a clean report also
291-    validates page stitching. Reports counts only ? no amounts.
292-    """
293-    entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in statement.transactions]
294-    return reconcile_running_balance(entries, tol)
295-
296-
297-def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
298-    """Pure core: read HDFC's SUMMARY block from extracted text lines.
299-
300-    Layout (stable across statements)::
301-
--
326-        return None
327-    bvals: list[float | None] = [*balances] if balances else [None, None, None, None]
328-    opening, debit_amt, credit_amt, closing = bvals[:4]
329-    cvals: list[int | None] = [*counts] if counts else [None, None]
330-    debit_count, credit_count = cvals[:2]
331-    return StatementSummary(
332-        opening_balance=opening,
333-        closing_balance=closing,
334-        debit_count=debit_count,
335-        credit_count=credit_count,
336-        debit_amount=debit_amt,
337-        credit_amount=credit_amt,
338-    )
339-
340-
341:def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
342-    """Read the bank's own SUMMARY figures from the statement, or None if absent."""
343-    with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
```

```
344-        for page in pdf.pages:
345-            text = page.extract_text() or ""
346-            if "SUMMARY" in text:
347-                summary = _parse_summary_lines(text.splitlines())
348-                if summary is not None:
349-                    return summary
350-    return None
351-
352-
353:def validate_against_summary(
354-    statement: Statement, summary: StatementSummary, tol: float = 0.01
355-) -> ValidationReport:
356-    """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.
357-
358-    Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
359-    (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
360-    error/omission, the Dr/Cr counts catch any extra/missing row, and the opening & closing
361-    balances catch a missing first/last row.
362-    """
363-    rows = [t.to_concept() for t in statement.transactions]
364-    return verify(rows, summary, tol)
365-
366-
367-def write_csv(statement: Statement, out_path: Path | str) -> Path:
368-    """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
```

10. assistant (2026-06-10T16:28:46.748Z)

The code-level claims check out so far. Now let me look for any mitigating tests, snapshot regression harness, or downstream checks that would catch the issue.

11. user (2026-06-10T16:28:50.302Z)

```
Found 7 files
tests\parsers\test_au.py
tests\parsers\test_axis.py
tests\parsers\test_hdfc.py
tests\parsers\test_validation.py
tests\parsers\test_engine.py
tests\test_ocr.py
tests\test_promo_regression.py
```

12. user (2026-06-10T16:28:50.358Z)

```
tests\test_smoke.py
tests\test_db.py
tests\test_extract.py
tests\test_promo_regression.py
tests\test_ocr.py
tests\test_unlock.py
tests\ingestion\test_gmail.py
tests\ingestion\test_providers.py
tests\test_offers.py
tests\test_repo_hygiene.py
tests\parsers\test_engine.py
tests\parsers\test_hdfc.py
tests\parsers\test_registry.py
tests\parsers\test_validation.py
tests\parsers\test_axis.py
tests\parsers\test_clustering.py
tests\parsers\test_local_axis.py
tests\parsers\test_au.py
tests\parsers\test_local_au.py
tests\synthpdf.py
```

```
tests\test_regression_synth.py
tests\parsers\test_savings.py
tests\test_eval_cli.py
tests\conftest.py
tests\test_cli.py
tests\test_db_crypto.py
tests\parsers\test_parse_errors.py
tests\test_errors.py
tests\test_log.py
tests\test_redaction.py
tests\test_password_rules.py
tests\test_secrets.py
tests\test_statement_unlock.py
tests\test_db_crypto_ops.py
tests\test_verifier_properties.py
tests\test_dependency_licenses.py
tests\test_precommit_hook.py
tests\test_recovery.py
tests\test_db_invariants.py
tests\test_migrations.py
tests\test_provenance.py
```

13. user (2026-06-10T16:28:55.802Z)

```
1       """Tests for the shared savings path's balance-delta reconstruction
(muneem.parsers.savings).
2
3       `_reconstruct_by_balance_delta` recovers debit/credit from the running-balance delta
when columns
4       don't split cleanly (a merged amount column, an interleaved value-date column, sparse
deposits). It
5       is exercised here on synthetic cell-matrices ? no PDF, no secrets ? including the
*dropped-row* case
6       (a balance jump matching no single amount) which must NOT reconcile (the real Axis May
2025 case).
7       """
8
9       from muneem.parsers import savings
10      from muneem.parsers.profiles import AXIS_PROFILE
11      from muneem.parsers.validation import reconcile_running_balance
12
13
14      def _matrix():
15          """June-style: date | narration | withdrawal(sparse) | deposit(sparse) | value-date
| balance.
16          Opening 10,000 ? 9,000 (?1,000) ? 14,000 (+5,000) ? 12,000 (?2,000)."""
17          return [
18              ["02-08-2025", "UPI-ABC", "1,000.00", "", "01-08-2025", "9,000.00"],
19              ["05-08-2025", "SALARY", "", "5,000.00", "04-08-2025", "14,000.00"],
20              ["10-08-2025", "ATM-CASH", "2,000.00", "", "09-08-2025", "12,000.00"],
21          ]
22
23
24      def _reconciles(rows):
25          return reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok
26
27
28      def test_reconstructs_direction_from_balance_delta_and_reconciles():
29          rows = savings._reconstruct_by_balance_delta(_matrix(), AXIS_PROFILE,
opening=10000.0)
30          assert len(rows) == 3
31          assert _reconciles(rows)
32          # direction derived from the sign of ?balance; the value-date column is ignored as a
flow
33          assert rows[0].debit == 1000.0 and rows[0].credit is None
```

```
34          assert rows[1].credit == 5000.0 and rows[1].debit is None
35          assert rows[2].debit == 2000.0 and rows[2].credit is None
36          assert [r.balance for r in rows] == [9000.0, 14000.0, 12000.0]
37
38
39      def test_a_dropped_transaction_does_not_reconcile():
40          # Drop a transaction (make the last balance jump): |?balance| no longer equals the
row's amount,
41          # so the reconstruction must NOT reconcile ? the oracle quarantines an incomplete
ledger (May).
42          m = _matrix()
43          m[2] = ["10-08-2025", "ATM-CASH", "2,000.00", "", "09-08-2025", "1,000.00"]
44          rows = savings._reconstruct_by_balance_delta(m, AXIS_PROFILE, opening=10000.0)
45          assert not _reconciles(rows)
46
47
48      def test_requires_opening_balance_to_fix_first_direction():
49          assert savings._reconstruct_by_balance_delta(_matrix(), AXIS_PROFILE, opening=None)
== []
50
51
52      def test_ignores_a_leading_non_transaction_noise_row():
53          # A header/summary line (no leading date) must be dropped, not parsed as a
transaction.
54          m = [["Opening Balance", "", "", "", "", "10,000.00"], *_matrix()]
55          rows = savings._reconstruct_by_balance_delta(m, AXIS_PROFILE, opening=10000.0)
56          assert len(rows) == 3 and _reconciles(rows)
57
```

14. user (2026-06-10T16:28:57.120Z)

```
tests\parsers\test_validation.py-77-def
test_verify_full_bookend_passes_and_reports_confidence():
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-78-    concepts =
StatementConcepts(
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-79-
opening_balance=10000.0, closing_balance=9999.5, debit_count=2, credit_count=1
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-80-    )
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-81-    r =
verify(_concept_rows(), concepts)
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-82-    assert r.reconciles and
r.ok
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-83-    assert r.debit_count_ok
and r.credit_count_ok and r.opening_ok and r.closing_ok
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-84-    assert r.confidence is
Confidence.RECONCILED
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-85-
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-86-
tests\parsers\test_validation.py:87:def
test_verify_unavailable_figures_are_not_checked_so_trust_is_withheld():
tests\parsers\test_validation.py-88-    # No SUMMARY figures: the per-row walk still passes, but
with nothing to bookend against the
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-89-    # parse is not *trusted*
(ok=False). This is why HDFC falls back to reconcile() when absent.
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-90-    r =
verify(_concept_rows(), StatementConcepts())
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-91-    assert r.reconciles
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-92-    assert r.debit_count_ok
is None and r.credit_count_ok is None
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-93-    assert r.opening_ok is
None and r.closing_ok is None
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-94-    assert not r.ok and
r.confidence is Confidence.UNRECONCILED
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-95-
C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py-96-
```

tests\parsers\test_validation.py-97-def test_verify_catches_wrong_count():

15. user (2026-06-10T16:29:08.021Z)

```
1       """Local-only checks for the Axis savings parser against real encrypted statements.
2
3       Skipped unless an Axis password is available (``password_rules.yaml`` or
4       ``MUNEEM_TEST_AXIS_PW``); never runs in CI. The savings parser is **oracle-gated**
5       (coordinate clustering + ``extract_tables`` ? engine mapping ? reconciliation), so this
6       asserts the safety invariant: it either stored nothing (a vintage it can't yet read) or
7       everything it stored reconciled ? never mislabeled rows. Surfaces counts / booleans /
8       parser names only ? no statement content.
9       """
10
11      import os
12      from pathlib import Path
13
14      import pdfplumber
15      import pytest
16
17      from muneem import db as dbmod
18      from muneem.parsers import get_parser
19      from muneem.password_rules import get_password_for_bank
20
21      AXIS_PDFS = sorted(Path("testdata/axis-statements").glob("*.pdf"))
22
23
24      @pytest.mark.local_statements
25      @pytest.mark.parametrize("pdf_path", AXIS_PDFS, ids=lambda p: p.name)
26      def test_local_axis_never_stores_unreconciled(pdf_path, tmp_path):
27          pw = get_password_for_bank("axis") or os.environ.get("MUNEEM_TEST_AXIS_PW")
28          if not pw:
29              pytest.skip("needs the Axis password (password_rules.yaml or
MUNEEM_TEST_AXIS_PW)")
30
31          with pdfplumber.open(pdf_path, password=pw) as pdf:
32              text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
len(pdf.pages))))
33          parser = get_parser(text)
34          if parser is None:
35              pytest.skip(f"{pdf_path.name}: no parser recognizes it yet (Axis rewrite
pending)")
36
37          with dbmod.connect(tmp_path / "t.db") as conn:
38              dbmod.init_db(conn)
39              doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
40              n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
41              status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
42              stored = dbmod.list_axis_transactions(conn)
43
44          # Oracle-gated invariant: the savings parser either stores nothing (and flags the
45          # document unreconciled) or, if it stored rows, they reconciled. Never garbage.
46          if type(parser).__name__ == "AxisParser":
47              assert (n == 0 and stored == []) or status == "reconciled", (
48                  f"{pdf_path.name}: stored {n} row(s) with status={status} ? "
49                  "stop-gap must not persist unreconciled rows"
50              )
51
```

16. user (2026-06-10T16:29:08.310Z)

```
1       """Layer-1 regression CI ? full-pipeline fixtures on synthetic born-digital PDFs.
2
3       Generates *real* PDFs (no committed statements, no secrets) and runs the production
```

```
pipeline ?
4       ``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert`` ? assert a **reconciled**
parse. This
5       covers the extraction / coordinate-clustering / engine layers end-to-end, the gap the
synthetic
6       word-list & table unit tests (which bypass pdfplumber) don't reach. See ROADMAP §
"Planned ?
7       Regression CI for statement parsing" (Layer 1). Always-on, secret-free ? runs in cloud
CI.
8       """
9
10      from pathlib import Path
11
12      from muneem import db as dbmod
13      from muneem.extract import extract
14      from muneem.parsers import get_parser
15      from synthpdf import at, build_pdf, right
16
17
18      def _hdfc_pdf(path: Path) -> Path:
19          """HDFC layout: leading date x0?52, narration x0?111, three right-aligned amount
columns whose
20          right edges land in the clusterer's bands (withdrawal <400, deposit <490, balance
?490), plus a
21          SUMMARY block for the bookend. Three rows reconcile (opening 10,000 ? closing
10,000)."""
22          p = [
23              at(50, 60, "HDFC BANK"),
24              at(50, 72, "Savings Account Details"),
25              at(50, 84, "Account Number : 50100012345678"),
26              # header ? must carry "Narration" + a "Closing" token so _table_body locates the
table
27              at(72, 110, "Date"),
28              at(175, 110, "Narration"),
29              at(300, 110, "Withdrawals"),
30              at(390, 110, "Deposits"),
31              at(480, 110, "Closing"),
32              at(515, 110, "Balance"),
33          ]
34          rows = [
35              ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
36              ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
37              ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
38          ]
39          top = 130
40          for d, narr, wd, dep, bal in rows:
41              p += [
42                  at(52, top, d),
43                  at(111, top, narr),
44                  right(358, top, wd),
45                  right(443, top, dep),
46                  right(561, top, bal),
47              ]
48              top += 15
49          p += [
50              at(50, 200, "SUMMARY"),
51              at(50, 212, "Opening Balance Debit Amount Credit Amount Closing Balance"),
52              at(50, 224, "10,000.00 2,000.00 2,000.00 10,000.00"),
53              at(50, 236, "Debit Count Credit Count"),
54              at(50, 248, "2 1"),
55          ]
56          return build_pdf(p, path)
57
58
59      def _au_savings_pdf(path: Path) -> Path:
```

```
60          """Savings layout (the shared savings.py engine path used by Axis/AU): a clean
61          date | narration | debit | credit | balance table in well-separated columns ?
coordinate
62          clustering ? engine mapping ? per-row reconcile. Three rows that walk (opening
50,000)."""
63          p = [
64              at(50, 60, "AU Small Finance Bank"),
65              at(50, 72, "Account Number : 1234567890123"),
66              # header (clustering's date+money filter drops it; here only for realism / text
markers)
67              at(52, 100, "Date"),
68              at(110, 100, "Transaction"),
69              at(310, 100, "Withdrawal"),
70              at(395, 100, "Deposit"),
71              at(485, 100, "Balance"),
72          ]
73          rows = [
74              ("02/08/2025", "UPI-ZOMATO", "1,200.00", "", "48,800.00"),
75              ("05/08/2025", "SALARY", "", "30,000.00", "78,800.00"),
76              ("10/08/2025", "ATM-CASH", "2,000.00", "", "76,800.00"),
77          ]
78          top = 124
79          for d, narr, debit, credit, bal in rows:
80              line = [at(52, top, d), at(110, top, narr), right(520, top, bal)]
81              if debit:
82                  line.append(right(340, top, debit))
83              if credit:
84                  line.append(right(420, top, credit))
85              p += line
86              top += 15
87          return build_pdf(p, path)
88
89
90      def _run_pipeline(pdf: Path) -> tuple[str, int, str]:
91          """Mirror `muneem ingest`: extract text ? pick parser ? parse+insert ? read back
status."""
92          ex = extract(pdf)
93          parser = get_parser(ex.text)
94          assert parser is not None, "no parser routed this synthetic statement"
95          with dbmod.connect(":memory:") as conn:
96              dbmod.init_db(conn)
97              doc_id = dbmod.insert_document(conn, source="file", sha256=ex.sha256,
status="parsed")
98              n = parser.parse_and_insert(pdf, conn, doc_id)
99              status = dbmod.get_document_by_sha(conn, ex.sha256)["status"]
100         return type(parser).__name__, n, status
101
102
103     def test_hdfc_layout_full_pipeline_reconciles(tmp_path):
104         name, n, status = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
105         assert name == "HDFCParser"
106         assert n == 3, f"expected 3 transactions, got {n}"
107         assert status == "reconciled", f"expected reconciled, got {status}"
108
109
110     def test_savings_layout_full_pipeline_reconciles(tmp_path):
111         name, n, status = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
112         assert name == "AUParser"
113         assert n == 3, f"expected 3 transactions, got {n}"
114         assert status == "reconciled", f"expected reconciled, got {status}"
115
```

17. user (2026-06-10T16:29:21.453Z)

```
1       """Tests for `muneem eval` ? the seed?golden onboarding harness.
```

```
2
3       Runs the CLI against a folder of *synthetic* statements (no secrets, no real data) and
checks the
4       reconcile-rate report. Reuses the born-digital fixtures from the regression suite.
5       """
6
7       from typer.testing import CliRunner
8
9       from muneem.cli import app
10      from test_regression_synth import _au_savings_pdf, _hdfc_pdf
11
12      runner = CliRunner()
13
14
15      def test_eval_reports_reconcile_rate(tmp_path):
16          _hdfc_pdf(tmp_path / "hdfc.pdf")
17          _au_savings_pdf(tmp_path / "au.pdf")
18          result = runner.invoke(app, ["eval", str(tmp_path)])
19          assert result.exit_code == 0, result.output
20          assert "2/2 reconciled" in result.output
21          assert "reconciled" in result.output  # per-file outcome line
22
23
24      def test_eval_empty_folder(tmp_path):
25          result = runner.invoke(app, ["eval", str(tmp_path)])
26          assert result.exit_code == 0
27          assert "no .pdf files" in result.output
28
```

18. user (2026-06-10T16:29:22.637Z)

```
Found 1 file
src\muneem\db.py
```

19. user (2026-06-10T16:29:29.494Z)

```
281-
282-
283-def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284-    """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285-
286:    Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL active,
so no
287-    naive file copy and no manual checkpoint needed. The destination is keyed with the same
key, so
288-    it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is wrong.
289-    """
290-    import sqlcipher3
291-
--
295-    source = _open_encrypted(str(db_path), key)  # validates key
296-    try:
297-        target = sqlcipher3.connect(str(dest_path))
298-        try:
299-            target.execute(_raw_key_pragma(key))  # key the destination so the backup is
encrypted
300:            source.backup(target)  # consistent snapshot of the live DB
301-        finally:
302-            target.close()
303-    finally:
304-        source.close()
305-    _harden_permissions(str(dest_path))
```

20. assistant (2026-06-10T16:29:36.473Z)

```
No mitigation there. Now let me empirically reproduce the claimed oracle failures with synthetic
numbers.
```

21. user (2026-06-10T16:29:45.546Z)

```
Exit code 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'muneem'
```

22. user (2026-06-10T16:29:51.109Z)

```
.venv/Scripts/python.exe
pyproject.toml
```

23. user (2026-06-10T16:30:02.485Z)

```
case a (missing leading half, implied opening 9000 vs printed 10000) endorsed: True
case b (first-row debit misread 9999) endorsed: True
case c (trailing txn missing, last balance 9400 vs closing 9000) endorsed: True
verify() case a ok: False
verify() case c ok: False
```

24. user (2026-06-10T16:30:43.400Z)

```
src\muneem\parsers\au.py:22:from .savings import SavingsParse, parse_savings
src\muneem\parsers\au.py:28:    return parse_savings(pdf_path, AUBANK_PROFILE, password)
src\muneem\parsers\axis.py:29:from .savings import SavingsParse, parse_savings,
parse_statement_summary, txn_line_predicate
src\muneem\parsers\axis.py:54:    """Parse an Axis savings statement via the shared engine-
driven path (savings.parse_savings)."""
src\muneem\parsers\axis.py:55:    return parse_savings(pdf_path, AXIS_PROFILE, password)
src\muneem\parsers\hdfc.py:364:    return verify(rows, summary, tol)
src\muneem\parsers\savings.py:176:def parse_savings(pdf_path: Path, profile: Profile, password:
str | None = None) -> SavingsParse:
src\muneem\parsers\savings.py:220:    "parse_savings",
src\muneem\parsers\validation.py:157:def verify(
```

25. user (2026-06-10T16:31:08.952Z)

```
Structured output provided successfully
```

26. assistant (2026-06-10T16:31:15.452Z)

```
Verification complete. The finding is confirmed as real with severity high justified.
```